CS-300 Analysis and Design

Project One Milestone Three: Tree Data Structure Pseudocode

Southern New Hampshire University

Joshua Evans

October 2, 2026

Overview

This pseudocode outlines the process for loading course data from a file, validating the data, creating course objects, storing them in a binary search tree, and printing course information. The design follows the requirements provided by ABC University and prepares the foundation for Project One.

Each line of the course file contains a course number, course name, and zero or more prerequisites. This pseudocode ensures the file is correctly formatted, validates that all prerequisites exist, and inserts each course into the binary search tree using the course number as the key.

Pseudocode

A1: Open File and Read Lines

```
BEGIN

    PROMPT user for course data file path
    OPEN file at given path
    IF file cannot be opened
        PRINT "Error: Cannot open file."
        STOP program
    ENDIF
```

A2: First Pass — Collect All Course Numbers

```
CREATE empty BinarySearchTree called courseTree
CREATE empty list called allCourseNumbers
FOR each line in file
    SPLIT line by commas into tokens
    IF number of tokens < 2
        PRINT "Format Error: Each line must contain at least a course number and a course name."
        CONTINUE to next line
    ENDIF

    SET courseNumber = tokens[0]
    ADD courseNumber to allCourseNumbers
ENDFOR
```

A3: Second Pass — Validate Prerequisites and Create Course Objects

```
RESET file pointer to beginning of file
FOR each line in file
        SPLIT line by commas into tokens
        SET courseNumber = tokens[0]
        SET courseName   = tokens[1]
        CREATE empty list prereqList
        FOR each token after the second token
                SET prereq = token
                IF prereq NOT IN allCourseNumbers
                        PRINT "Error: Prerequisite " + prereq + " does not exist in file."
                        CONTINUE to next line
                ENDIF

                ADD prereq to prereqList
        ENDFOR

        CREATE new Course object
        SET Course.number = courseNumber
        SET Course.name   = courseName
        SET Course.prerequisites = prereqList
        INSERT Course object into courseTree using courseNumber as key
ENDFOR
END
```

B1: Course Structure Definition

```
STRUCT Course
    STRING number
    STRING name
    LIST prerequisites
END STRUCT
```

C1: Insert Course Into Binary Search Tree

```
FUNCTION insertCourse(courseTree, course)
    IF courseTree.root is null
        courseTree.root = new Node(course)
    ELSE
        CALL addNode(courseTree.root, course)
    ENDIF
END FUNCTION
```

C2: Recursive Node Insertion

```
FUNCTION addNode(node, course)
    key = course.number
    nodeKey = node.course.number
    IF key < nodeKey
        IF node.left is null
            node.left = new Node(course)
```

```
            ELSE
                CALL addNode(node.left, course)
            ENDIF
        ELSE
            IF node.right is null
                node.right = new Node(course)
            ELSE
                CALL addNode(node.right, course)
            ENDIF
        ENDIF

END FUNCTION
```

D1: Print a Single Course and Its Prerequisites

```
FUNCTION printCourse(courseTree, courseNumber)
    SET current = courseTree.root
    WHILE current is not null
        IF current.course.number == courseNumber
            PRINT current.course.number + ": " + current.course.name

            IF current.course.prerequisites list is empty
                PRINT "  No prerequisites."
            ELSE
                PRINT "  Prerequisites:"
                FOR each prereq in current.course.prerequisites
```

```
                    PRINT "    " + prereq
                ENDFOR
            ENDIF
            RETURN
        ENDIF
        IF courseNumber < current.course.number
            current = current.left
        ELSE
            current = current.right
        ENDIF

    ENDWHILE

    PRINT "Course " + courseNumber + " not found."

END FUNCTION
```

D2: Print All Courses (In-Order Traversal)

```
FUNCTION printAllCourses(courseTree)
    CALL inOrder(courseTree.root)
END FUNCTION

FUNCTION inOrder(node)
    IF node is null
        RETURN
```

```
        ENDIF
        CALL inOrder(node.left)
        PRINT node.course.number + ": " + node.course.name

        IF node.course.prerequisites list is empty
                PRINT "  No prerequisites."
        ELSE
                PRINT "  Prerequisites:"
                FOR each prereq in node.course.prerequisites
                        PRINT "    " + prereq
                ENDFOR
        ENDIF
        CALL inOrder(node.right)
END FUNCTION
```